

Computer Architecture and Assembly Language

Kalman Filter: Milestone 3

RISC-V Scalar Assembly Implementation

Team Warjingo

Muhammad Ismail	31689
Mehdi Ali	30507
Muhammad Ismail	30917
Arham Awan	30934

ABSTRACT

This report covers the RISC-V scalar assembly implementation of two state estimation filters—the Linear Kalman Filter (LKF) and the Extended Kalman Filter (EKF)—for a 23-joint 3D human gait tracking system. The Milestone 2 C++ reference code was translated function-by-function into hand-written RV64GD assembly, with all computation performed in assembly and only CSV I/O delegated to minimal C helper files.

Both assembly implementations were validated against the C++ reference across all 3040 timesteps and 276 state dimensions. The LKF and EKF each achieved a maximum absolute error of exactly 0.0, confirming bit-identical numerical output. This was accomplished by matching IEEE 754 rounding behavior through careful use of fused multiply-add (`fmadd.d`) and fused negative-multiply-subtract (`fnmsub.d`) instructions at the same computational points as the C++ reference.

The report walks through each assembly function with code excerpts, documents the register allocation strategy, presents the numerical verification results in tabular form, and includes a full suite of 12 plots comparing assembly output against measurements and ground truth.

Contents

1	Introduction	3
1.1	Source Files	3
1.2	Build and Verification	3
2	System Overview	3
3	Design Choices	4
3.1	Handle Table Architecture	4
3.2	Floating-Point Constants	4
4	Function-by-Function Walkthrough	4
4.1	<code>vec_add</code> and <code>vec_sub</code>	5
4.2	<code>transpose_mat</code>	5
4.3	<code>multiply_mat</code> — Matrix Multiplication	6
4.4	<code>solve_system</code> — LU Decomposition with Partial Pivoting	8
4.5	<code>approx_atan</code> and <code>approx_atan2</code> (EKF only)	9
4.6	<code>state_to_spherical</code> and <code>meas_to_spherical</code> (EKF only)	11
4.7	<code>compute_jac</code> (EKF only)	12
5	Matrix Inversion Strategy	13
6	Memory Management	13
6.1	Heap Allocation	13
6.2	Workspace in <code>solve_system</code>	14
7	Numerical Verification	14
7.1	Why the Error Is Exactly Zero	14
7.2	Per-State-Component Breakdown	14
7.3	Per-Joint Verification	15
8	Plots and Analysis	15
8.1	Time-Series Plots	15
8.2	True vs Noisy vs Estimated Position	18
8.3	LKF vs EKF Comparison	19
8.4	Milestone-2 vs Milestone-3 Comparison	22
9	Instruction Usage Summary	24
10	Conclusion	25
	References	25

1. Introduction

The goal of Milestone 3 is to re-implement the LKF and EKF—originally written in C++ for Milestone 2—in RISC-V scalar assembly. The system tracks 23 joints of a full-body walking dataset using a constant-jerk motion model. Each joint carries 12 state variables (position, velocity, acceleration, jerk along x, y, z), giving a state dimension of $N = 276$ and a measurement dimension of $M = 69$.

Writing a Kalman Filter in assembly involves managing large heap-allocated matrices (up to 276×276), respecting the RISC-V calling convention, and dealing with the limited register file. The numerical requirement is strict: every output value must agree with the C++ reference to within 10^{-9} .

1.1 Source Files

Two assembly files are produced:

- `lkf_asm.s` — LKF with all shared math functions (`vec_add`, `multiply_mat`, `solve_system`, etc.) and the complete filter loop.
- `ekf_asm.s` — EKF with additional functions for the nonlinear measurement model: `approx_atan`, `approx_atan2`, `state_to_spherical`, `meas_to_spherical`, and `compute_jac`.

Two C helper files (`lkf_helpers.c`, `ekf_helpers.c`) provide CSV I/O through `load_csv`, `csv_open`, `csv_write_state`, and `csv_close`. No computation is done in C.

1.2 Build and Verification

A Makefile handles everything:

```
make clean && make run
```

This assembles both `.s` files, links them with the C helpers, runs both filters under `gemu-riscv64`, compiles and runs the C++ reference, and automatically compares all outputs via `verify_all`.

2. System Overview

The state vector, transition matrix, and noise parameters carry over from Milestone 2:

$$N = 276, \quad M = 69, \quad \Delta t = 0.01 \text{ s}, \quad \sigma_j = 1.0, \quad \sigma_r = 0.5$$

$$\mathbf{F} \in \mathbb{R}^{276 \times 276}, \quad \mathbf{Q} \in \mathbb{R}^{276 \times 276}, \quad \mathbf{H} \in \mathbb{R}^{69 \times 276}, \quad \mathbf{R} \in \mathbb{R}^{69 \times 69}$$

$\mathbf{F}$ is block-diagonal: 23 copies of a 12×12 joint block, each containing three 4×4 constant-jerk blocks. $\mathbf{Q}$ has the same structure. These matrices are over 99% zeros, which we exploit through a zero-skipping optimization in matrix multiplication.

For the EKF, $\mathbf{R}$ has a different diagonal structure: 0.25 for range measurements and 0.01 for azimuth and elevation, reflecting the different noise characteristics of spherical coordinates.

3. Design Choices

3.1 Handle Table Architecture

Storing 33 matrix pointers in registers is impossible with only 12 callee-saved registers. Our solution is a *handle table*: a single heap-allocated array of 33 pointers, with register `s3` holding its base address throughout the filter loop. Any matrix pointer is loaded with one instruction:

```
1 .equ H_F,      0      # F matrix handle
2 .equ H_Q,      8      # Q matrix handle
3 .equ H_st,     24     # state vector handle
4 .equ H_cv,     32     # covariance handle
5 ...
6      ld        a0, H_F(s3) # load F pointer
```

The five registers used in main:

tabheadblue	
<code>s0</code>	Base address of noisy data (from <code>load_csv</code>)
<code>s1</code>	Number of frames $T = 3040$
<code>s2</code>	Columns per frame (69)
<code>s3</code>	Handle table base pointer
<code>s4</code>	Frame counter t

3.2 Floating-Point Constants

LKF constants (Δt , 0.5, 1/6, 1.0) are loaded via their IEEE 754 hex representation:

```
1      li        t0, 0x3F847AE147AE147B    # 0.01
2      fmv.d.x   ft0, t0
```

EKF trigonometric constants are stored in `.rodata` with `.double` directives to guarantee exact representation:

```
1      .section .rodata
2      c_pio4:    .double 0.7853981633974483
3      c_a1:      .double 0.2447
4      c_a2:      .double 0.0663
5      c_pi:      .double 3.14159265358979323846
6      c_halfpi:  .double 1.5707963267948966
7      c_epsilon: .double 1.0e-9
```

4. Function-by-Function Walkthrough

This section walks through every assembly function in both `lkf_asm.s` and `ekf_asm.s`. For each function we show the actual code, explain the instruction choices, and document which registers are used and why.

4.1 vec_add and vec_sub

These are the simplest computational functions. Both take four arguments via the RISC-V calling convention: `a0` = pointer to source *A*, `a1` = pointer to source *B*, `a2` = pointer to destination *C*, and `a3` = total element count. They iterate linearly over all elements, performing `fadd.d` or `fsub.d` on each pair. No callee-saved registers are touched, so no stack frame or prologue/epilogue is needed—the function uses only temporaries `t0–t4` and FP temporaries `ft0–ft2`.

```
1  vec_add:
2      li      t0, 0                # i = 0 (loop counter)
3      .Lva_lp:
4          bge   t0, a3, .Lva_dn     # if i >= len, we are done
5          slli  t1, t0, 3           # byte offset = i * 8 (double)
6          add   t2, a0, t1          # compute &A[i]
7          add   t3, a1, t1          # compute &B[i]
8          add   t4, a2, t1          # compute &C[i]
9          fld   ft0, 0(t2)          # load A[i] into ft0
10         fld   ft1, 0(t3)          # load B[i] into ft1
11         fadd.d ft2, ft0, ft1       # ft2 = A[i] + B[i]
12         fsd   ft2, 0(t4)          # store result to C[i]
13         addi  t0, t0, 1           # i++
14         j     .Lva_lp            # repeat
15     .Lva_dn:
16         ret                      # return to caller
```

`vec_sub` is identical in structure but uses `fsub.d` instead of `fadd.d`. These functions correspond to the `operator+` and `operator-` in the C++ `Matrix` class. They are called for operations like $P_{\text{pred}} = FPF^T + Q$ (addition) and $y = z - Hx$ (subtraction), as well as $I - KH$ in the Joseph-form covariance update.

4.2 transpose_mat

Transposes an $r \times c$ matrix *A* into a $c \times r$ matrix *B*. The mapping is $(i, j) \rightarrow (j, i)$: source element at row-major index $i \cdot c + j$ goes to destination index $j \cdot r + i$. Two nested loops iterate over rows and columns, with `mul` computing the row-stride offsets and `slli` by 3 converting element indices to byte offsets. Only temporary registers are used, so no stack frame is allocated.

```

1 transpose_mat:
2     li      t0, 0                # i = 0 (row counter)
3 .Ltm_r:
4     bge     t0, a2, .Ltm_dn      # if i >= nrow, done
5     li      t1, 0                # j = 0 (column counter)
6 .Ltm_c:
7     bge     t1, a3, .Ltm_nr      # if j >= ncol, next row
8     mul     t2, t0, a3           # src offset: i * ncol
9     add     t2, t2, t1           # + j
10    slli    t2, t2, 3            # * 8 bytes per double
11    add     t3, a0, t2           # &src[i*ncol + j]
12    mul     t4, t1, a2           # dst offset: j * nrow
13    add     t4, t4, t0           # + i
14    slli    t4, t4, 3            # * 8 bytes
15    add     t5, a1, t4           # &dst[j*nrow + i]
16    fld     ft0, 0(t3)           # load src element
17    fsd     ft0, 0(t5)           # store to transposed position
18    addi    t1, t1, 1            # j++
19    j       .Ltm_c
20 .Ltm_nr:
21    addi    t0, t0, 1            # i++
22    j       .Ltm_r
23 .Ltm_dn:
24    ret

```

This function is called frequently: once for F^T , once for H^T (or J^T in EKF), once for $(PH^T)^T$ during gain computation, once for K^T during the covariance update, and once for $(I - KH)^T$.

4.3 multiply_mat — Matrix Multiplication

This is the most performance-critical function in the entire implementation. It is called approximately 15 times per timestep for operations like $F \cdot x$, $F \cdot P$, $FP \cdot F^T$, $H \cdot P$, $HP \cdot H^T$, $K \cdot y$, $(I - KH) \cdot P$, and the remaining terms of the Joseph-form covariance update. With 3040 timesteps, that amounts to roughly 45,000 calls during a single filter run.

The function takes six arguments: pointers to matrices A , B , C and their dimensions (rowsA, colsA, colsB). Because all six arguments must survive across the nested loops, they are moved into callee-saved registers s0–s5, requiring a 48-byte stack frame:

```

1 multiply_mat:
2     addi    sp, sp, -48          # allocate 48 bytes on stack
3     sd      s0, 0(sp)           # save s0 (will hold A ptr)
4     sd      s1, 8(sp)           # save s1 (will hold B ptr)
5     sd      s2, 16(sp)          # save s2 (will hold C ptr)
6     sd      s3, 24(sp)          # save s3 (will hold rowsA)
7     sd      s4, 32(sp)          # save s4 (will hold colsA)
8     sd      s5, 40(sp)          # save s5 (will hold colsB)
9     mv      s0, a0              # s0 = A pointer
10    mv      s1, a1              # s1 = B pointer
11    mv      s2, a2              # s2 = C pointer (output)
12    mv      s3, a3              # s3 = number of rows of A
13    mv      s4, a4              # s4 = number of columns of A
14    mv      s5, a5              # s5 = number of columns of B

```

The output matrix C is first zeroed, then we use the ikj loop ordering. The two key design decisions are:

Zero-skipping for sparse matrices. Before entering the innermost j -loop, $A[i][k]$ is loaded and compared against 0.0. If the element is zero, the entire inner loop (which would iterate over all columns of B) is skipped. This optimization is critical because the system matrices $\mathbf{F}$ (276×276) and $\mathbf{H}$ (69×276) are over 99% zeros—they are block-diagonal with only a handful of nonzero entries per row. Without this check, multiplying two 276×276 matrices would require $276^3 \approx 21$ million multiply-add operations. With zero-skipping, only the nonzero entries contribute, reducing the effective work by two orders of magnitude and making the filter feasible under QEMU emulation.

1	<code>fld</code>	<code>ft0, 0(t3)</code>	<code># ft0 = A[i][k]</code>
2	<code>fmv.d.x</code>	<code>ft7, zero</code>	<code># ft7 = 0.0 (IEEE 754 +0)</code>
3	<code>feq.d</code>	<code>a6, ft0, ft7</code>	<code># set a6=1 if A[i][k] == 0.0</code>
4	<code>bnez</code>	<code>a6, .Lmm_nm</code>	<code># if zero, skip entire j-loop</code>

The `fmv.d.x ft7, zero` instruction moves the integer zero register (`x0`, which is hardwired to 0) into FP register `ft7` as a raw 64-bit pattern. Since the all-zeros bit pattern is IEEE 754 positive zero, this gives us `+0.0` for comparison. The `feq.d` instruction then performs an exact floating-point equality test.

Fused multiply-add. The accumulation $C[i][j] += A[i][k] \cdot B[k][j]$ uses the `fmadd.d` instruction. This computes the product and sum in a single operation with only one rounding step, rather than the two rounding steps that separate `fmul.d` + `fadd.d` would produce. The Milestone 3 specification explicitly requires `fmadd.d` wherever a multiply-accumulate pattern appears.

1	<code>fld</code>	<code>ft1, 0(a7)</code>	<code># ft1 = B[k][j]</code>
2	<code>fld</code>	<code>ft2, 0(a7)</code>	<code># ft2 = C[i][j] (current sum)</code>
3	<code>fmadd.d</code>	<code>ft2, ft0, ft1, ft2</code>	<code># ft2 = A[i][k]*B[k][j] + C[i][j]</code>
4	<code>fsd</code>	<code>ft2, 0(a7)</code>	<code># store updated C[i][j]</code>

Note that `ft0` (holding $A[i][k]$) is loaded once in the outer k -loop and reused across the entire inner j -loop without reloading. This reduces memory traffic significantly.

Register allocation for `multiply_mat`:

tabheadblue		
<code>s0</code>	Source matrix A base pointer	Yes (stack)
<code>s1</code>	Source matrix B base pointer	Yes (stack)
<code>s2</code>	Destination matrix C base pointer	Yes (stack)
<code>s3</code>	Row count of A (<code>rowsA</code>)	Yes (stack)
<code>s4</code>	Inner dimension (<code>colsA</code> = <code>rowsB</code>)	Yes (stack)
<code>s5</code>	Column count of B (<code>colsB</code>)	Yes (stack)
<code>t0</code>	Outer loop counter i	Temporary
<code>t1</code>	Middle loop counter k	Temporary
<code>t6</code>	Inner loop counter j	Temporary
<code>t4, t5</code>	Precomputed row offsets ($k \cdot \text{colsB}$, $i \cdot \text{colsB}$)	Temporary
<code>ft0</code>	$A[i][k]$ (loaded once, reused across j)	Temporary
<code>ft1</code>	$B[k][j]$ (loaded each j iteration)	Temporary
<code>ft2</code>	$C[i][j]$ accumulator	Temporary
<code>ft3</code>	Zero constant for clearing C	Temporary
<code>ft7</code>	Zero constant for sparsity comparison	Temporary

4.4 solve_system — LU Decomposition with Partial Pivoting

This function solves the linear system $AX = B$ without computing A^{-1} explicitly. It is the most register-intensive function in the implementation, requiring all 13 callee-saved registers (`ra`, `s0–s11`) and a 112-byte stack frame. The function allocates four temporary heap buffers and frees them before returning.

The prologue saves all registers and copies the five arguments into callee-saved registers:

```

1 solve_system:
2     addi    sp, sp, -112           # 14 * 8 = 112 bytes
3     sd      ra, 0(sp)             # save return address
4     sd      s0, 8(sp)             # will hold A pointer
5     sd      s1, 16(sp)            # will hold B pointer
6     sd      s2, 24(sp)            # will hold X pointer (output)
7     sd      s3, 32(sp)            # will hold n (matrix size)
8     sd      s4, 40(sp)            # will hold m (RHS columns)
9     sd      s5, 48(sp)            # will hold LU workspace
10    ...                           # s6-s11 saved at offsets 56-96
11    mv      s0, a0                 # s0 = coefficient matrix A
12    mv      s1, a1                 # s1 = right-hand side B
13    mv      s2, a2                 # s2 = solution output X
14    mv      s3, a3                 # s3 = n (dimension)
15    mv      s4, a4                 # s4 = m (number of RHS cols)

```

Algorithm. The implementation follows the same strategy as the C++ `Matrix::solve`:

1. **Copy A into LU :** Heap-allocate an $n \times n$ workspace via `malloc` and copy A element-by-element. This preserves the original A since LU factorization is destructive.
2. **Partial pivoting:** For each column k , scan rows k through $n - 1$ to find the row with the largest $|LU[i][k]|$. Absolute value is computed using `fsgnjx.d ft1, ft0, ft0` (sign injection with XOR produces $|x|$). Comparisons use `fle.d`. If a better pivot is found, swap rows in both LU and the pivot index array.
3. **Gaussian elimination:** Divide $LU[i][k]$ by the pivot using `fdiv.d`, then update the remaining entries: $LU[i][j] -= LU[i][k] \cdot LU[k][j]$. This subtraction-after-multiply is performed using `fnmsub.d`.
4. **Permute B :** Reorder the rows of B according to the pivot array, storing the result in workspace B_p .
5. **Forward substitution:** Solve $LY = B_p$ using `fnmsub.d` for the inner-product accumulation $s -= LU[i][k] \cdot Y[k][j]$.
6. **Back substitution:** Solve $UX = Y$ using `fnmsub.d` and final division by $LU[i][i]$ via `fdiv.d`.
7. **Free workspace:** All four heap allocations (LU , `piv`, B_p , Y) are freed before returning.

The `fnmsub.d` instruction. The RISC-V instruction `fnmsub.d rd, rs1, rs2, rs3` computes $rs3 - rs1 \times rs2$ with a single rounding step. This is exactly the pattern $x \leftarrow x - a \cdot b$ needed in Gaussian elimination and in both substitution phases. The instruction appears three times in `solve_system`: once in the elimination inner loop, once in forward substitution, and once in back substitution.

```

1  # Elimination inner loop: LU[i][j] -= LU[i][k] * LU[k][j]
2      fld      ft2, 0(t6)          # ft2 = LU[i][j]
3      fld      ft3, 0(a7)          # ft3 = LU[k][j]
4      fnmsub.d ft2, ft0, ft3, ft2 # ft2 = LU[i][j] - LU[i][k]*LU[k][j]
5      fsd      ft2, 0(t6)          # store back
6
7  # Forward substitution: s -= LU[i][k] * Y[k][j]
8      fld      ft1, 0(t6)          # ft1 = LU[i][k]
9      fld      ft2, 0(a7)          # ft2 = Y[k][j]
10     fnmsub.d ft0, ft1, ft2, ft0 # ft0 = s - LU[i][k]*Y[k][j]

```

Register allocation for solve_system:

tabheadblue		
s0–s4	Input arguments (A, B, X, n, m)	Entire function
s5–s8	Heap workspace (LU, piv, B_p, Y)	Post-allocation
s9	Column counter k (factorization phase)	Factorization
s10	Pivot row index (search phase)	Pivot search
s11	Row counter i (elimination phase)	Elimination
ft0–ft5	Matrix elements, pivot values, accumulators	Temporary

4.5 approx_atan and approx_atan2 (EKF only)

The EKF measurement model converts Cartesian positions to spherical coordinates (r, θ, ϕ) , which requires the `atan2` function. As required by the milestone specification, this is implemented entirely in assembly without calling any math library.

Polynomial approximation. The `approx_atan` function computes the arctangent for $|z| \leq 1$ using the corrected fast approximate formula:

$$\text{atan}(z) \approx \frac{\pi}{4} z - z (|z| - 1) (0.2447 + 0.0663 |z|)$$

This requires only basic arithmetic: one absolute value (`fsgnjb.d`), four multiplications, two additions, and one subtraction. The maximum approximation error is about 0.0016 radians (0.09°), which is negligible compared to the measurement noise ($\sigma_\theta = 0.1$ rad).

```

1 approx_atan:
2     fsgnjx.d ft0, fa0, fa0      # ft0 = |z| (abs via sign inject)
3     la      t0, c_pio4
4     fld     ft1, 0(t0)         # ft1 = pi/4
5     la      t0, c_a1
6     fld     ft2, 0(t0)         # ft2 = 0.2447
7     la      t0, c_a2
8     fld     ft3, 0(t0)         # ft3 = 0.0663
9     la      t0, c_one
10    fld     ft4, 0(t0)         # ft4 = 1.0
11    fmul.d   ft5, ft1, fa0      # ft5 = (pi/4) * z
12    fsub.d   ft6, ft0, ft4      # ft6 = |z| - 1
13    fmul.d   ft7, ft3, ft0      # ft7 = 0.0663 * |z|
14    fadd.d   ft7, ft2, ft7      # ft7 = 0.2447 + 0.0663*|z|
15    fmul.d   ft6, fa0, ft6      # ft6 = z * (|z| - 1)
16    fmul.d   ft6, ft6, ft7      # ft6 = z*(|z|-1)*(0.2447+...)
17    fsub.d   fa0, ft5, ft6      # result = pi/4*z - correction
18    ret

```

Quadrant handling in approx_atan2. The full `atan2(y, x)` function reduces the problem so that `approx_atan` always receives an argument with $|z| \leq 1$:

- If $x = 0$ and $y = 0$: return 0.
- If $x = 0$: return $\pm\pi/2$ based on the sign of y .
- If $|x| \geq |y|$: compute $\text{atan}(y/x)$, which satisfies $|y/x| \leq 1$. Adjust by $\pm\pi$ if $x < 0$.
- If $|y| > |x|$: compute $\pi/2 - \text{atan}(x/y)$, which satisfies $|x/y| \leq 1$. Negate if $y < 0$, and adjust by $\pm\pi$ if needed.

Critical register strategy. The inner call to `approx_atan` clobbers all temporary FP registers (`ft0`–`ft7`). If y , x , π , or $\pi/2$ were stored in temporaries, they would be silently destroyed. We solve this by using callee-saved FP registers `fs0`–`fs3`, which are preserved across function calls:

```

1 approx_atan2:
2     addi     sp, sp, -40
3     sd      ra, 0(sp)         # save return address
4     fsd     fs0, 8(sp)        # save/restore callee-saved FP
5     fsd     fs1, 16(sp)
6     fsd     fs2, 24(sp)
7     fsd     fs3, 32(sp)
8     fmv.d   fs0, fa0          # fs0 = y (survives atan call)
9     fmv.d   fs1, fa1          # fs1 = x (survives atan call)
10    la      t0, c_pi
11    fld     fs2, 0(t0)         # fs2 = pi (survives atan call)
12    la      t0, c_halfpi
13    fld     fs3, 0(t0)         # fs3 = pi/2 (survives call)

```

This was one of the subtler bugs encountered during development. Our initial version stored π and $\pi/2$ in temporary registers `ft5` and `ft6`. The code appeared to work on simple test cases, but the EKF estimates gradually drifted after a few hundred timesteps. The root cause was that `approx_atan`'s internal computation overwrote `ft5` and `ft6`, and the subsequent quadrant correction used garbage values for π .

Register allocation for approx_atan2:

tabheadblue		
fs0	y input (preserved across <code>atan</code> call)	Yes (stack)
fs1	x input (preserved across <code>atan</code> call)	Yes (stack)
fs2	π constant (preserved across call)	Yes (stack)
fs3	$\pi/2$ constant (preserved across call)	Yes (stack)
fa0	Input argument / return value	Arg/Ret
ft0–ft7	Scratch computations (destroyed by inner call)	Temporary

4.6 state_to_spherical and meas_to_spherical (EKF only)

These two functions convert Cartesian positions to spherical coordinates (r, θ, ϕ) for all 23 joints. They share the same mathematical computation but differ in how they read the input data:

- `state_to_spherical` reads from the 276-element state vector. For each joint, p_x is at byte offset 0, p_y at offset 32 (4×8 , because velocity, acceleration, and jerk for the x -axis sit between the two position components), and p_z at offset 64 (8×8).
- `meas_to_spherical` reads from the 69-element flat measurement vector, where positions are stored as consecutive (p_x, p_y, p_z) triples with no gaps.

Both functions use callee-saved FP registers `fs0–fs2` to hold p_x, p_y, p_z across the two calls to `approx_atan2` (one for $\theta = \text{atan2}(p_y, p_x)$ and one for $\phi = \text{atan2}(p_z, \rho)$):

```

1 state_to_spherical:
2     ...
3     fld     fs0, 0(t2)           # px (callee-saved)
4     fld     fs1, 32(t2)          # py (offset 4*8 = 32 bytes)
5     fld     fs2, 64(t2)          # pz (offset 8*8 = 64 bytes)
6     # compute r = sqrt(px^2 + py^2 + pz^2)
7     fmul.d  ft0, fs0, fs0         # px^2
8     fmul.d  ft1, fs1, fs1         # py^2
9     fmul.d  ft2, fs2, fs2         # pz^2
10    fadd.d  ft0, ft0, ft1         # px^2 + py^2
11    fadd.d  ft0, ft0, ft2         # + pz^2
12    fsqrt.d ft3, ft0              # r = sqrt(...)
13    fsd     ft3, 0(t2)           # store r
14    # theta = atan2(py, px) -- fs0, fs1 survive the call
15    fmv.d   fa0, fs1             # y argument = py
16    fmv.d   fa1, fs0             # x argument = px
17    call    approx_atan2         # returns theta in fa0
18    fsd     fa0, 8(t2)           # store theta
19    # rho = sqrt(px^2 + py^2)
20    fmul.d  ft0, fs0, fs0         # px^2 (fs0 survived!)
21    fmul.d  ft1, fs1, fs1         # py^2 (fs1 survived!)
22    fadd.d  ft0, ft0, ft1
23    fsqrt.d ft3, ft0             # rho
24    # phi = atan2(pz, rho) -- fs2 survived both calls
25    fmv.d   fa0, fs2             # y argument = pz
26    fmv.d   fa1, ft3             # x argument = rho
27    call    approx_atan2         # returns phi in fa0
28    fsd     fa0, 16(t2)          # store phi
29    ...

```

Register allocation for `state_to_spherical`:

tabheadblue		
s0	Pointer to state vector x	Yes (stack)
s1	Pointer to output spherical array	Yes (stack)
s2	Joint loop counter (0...22)	Yes (stack)
fs0	p_x (survives both <code>atan2</code> calls)	Yes (stack)
fs1	p_y (survives both <code>atan2</code> calls)	Yes (stack)
fs2	p_z (survives both <code>atan2</code> calls)	Yes (stack)

4.7 compute_jac (EKF only)

Computes the 69×276 Jacobian matrix $\mathbf{J}$ of partial derivatives of the spherical measurement function with respect to the state vector. Out of the $3 \times 12 = 36$ possible entries per joint, only 8 are nonzero (the three position variables affect all three spherical coordinates, minus one zero entry for $\partial\theta/\partial p_z = 0$). The function first zeros the entire 69×276 matrix via `clear_doubles`, then fills in only the nonzero analytical derivatives:

$$\begin{aligned} \frac{\partial r}{\partial p_x} &= \frac{p_x}{r}, & \frac{\partial r}{\partial p_y} &= \frac{p_y}{r}, & \frac{\partial r}{\partial p_z} &= \frac{p_z}{r} \\ \frac{\partial \theta}{\partial p_x} &= \frac{-p_y}{\rho^2}, & \frac{\partial \theta}{\partial p_y} &= \frac{p_x}{\rho^2} \\ \frac{\partial \phi}{\partial p_x} &= \frac{-p_x p_z}{r^2 \rho}, & \frac{\partial \phi}{\partial p_y} &= \frac{-p_y p_z}{r^2 \rho}, & \frac{\partial \phi}{\partial p_z} &= \frac{\rho}{r^2} \end{aligned}$$

where $r = \sqrt{p_x^2 + p_y^2 + p_z^2}$ and $\rho = \sqrt{p_x^2 + p_y^2}$.

A guard condition prevents division by zero by checking both r and ρ against 10^{-9} before computing any derivatives for a given joint. This matches the C++ behavior exactly:

```

1      fld      ft0, 0(t2)          # px
2      fld      ft1, 32(t2)         # py
3      fld      ft2, 64(t2)         # pz
4      fmul.d   ft3, ft0, ft0        # px^2
5      fmul.d   ft4, ft1, ft1        # py^2
6      fmul.d   ft5, ft2, ft2        # pz^2
7      fadd.d   ft6, ft3, ft4        # rho^2 = px^2 + py^2
8      fadd.d   ft7, ft6, ft5        # r^2 = px^2 + py^2 + pz^2
9      fsqrt.d  ft8, ft7             # r
10     fsqrt.d  ft9, ft6             # rho
11     la       t0, c_epsilon
12     fld      ft10, 0(t0)          # epsilon = 1e-9
13     flt.d    t0, ft8, ft10         # is r < epsilon?
14     bnez     t0, .Lcj_nx          # skip this joint entirely
15     flt.d    t0, ft9, ft10         # is rho < epsilon?
16     bnez     t0, .Lcj_nx          # skip this joint entirely
17     # -- safe to compute derivatives --
18     fdiv.d   ft3, ft0, ft8        # dr/dpx = px / r
19     ...
20     fsgnjd   ft3, ft1, ft1        # -py (negate via sign inject)
21     fdiv.d   ft3, ft3, ft6        # dtheta/dpx = -py / rho^2
22     ...

```

The `fsgnjd` instruction negates a value by injecting the opposite sign bit, which is more efficient than loading -1.0 and multiplying.

Register allocation for `compute_jac`:

tabheadblue		
<code>s0</code>	Pointer to state vector x	Yes (stack)
<code>s1</code>	Pointer to output Jacobian J (69×276)	Yes (stack)
<code>s2</code>	Joint loop counter ($0 \dots 22$)	Yes (stack)
<code>s3</code>	Row base index ($j \times 3$) for the 3 measurement rows	Yes (stack)
<code>s4</code>	Column base index ($j \times 12$) for the 12 state columns	Yes (stack)
<code>ft0–ft2</code>	p_x, p_y, p_z position components	Temporary
<code>ft3–ft5</code>	p_x^2, p_y^2, p_z^2	Temporary
<code>ft6</code>	$\rho^2 = p_x^2 + p_y^2$	Temporary
<code>ft7</code>	$r^2 = p_x^2 + p_y^2 + p_z^2$	Temporary
<code>ft8</code>	$r = \sqrt{r^2}$	Temporary
<code>ft9</code>	$\rho = \sqrt{\rho^2}$	Temporary
<code>ft10</code>	$\epsilon = 10^{-9}$ guard threshold	Temporary

5. Matrix Inversion Strategy

No explicit matrix inverse is computed. The Kalman gain K is obtained by solving:

$$S \cdot K^T = (P \cdot J^T)^T$$

1. Compute PJ^T via `multiply_mat` (276×69).
2. Transpose to get $(PJ^T)^T$ (69×276) via `transpose_mat`.
3. Solve $S \cdot X = (PJ^T)^T$ using `solve_system`, where S is 69×69 .
4. Transpose the result to obtain K (276×69).

6. Memory Management

6.1 Heap Allocation

All matrices are heap-allocated via `malloc` called from assembly. A single 276×276 matrix occupies $276^2 \times 8 \approx 595$ KB. With 33 matrices needed simultaneously, heap allocation is the only option.

tabheadblue		
F, Q, P, P_{pred} , etc.	276×276	595 each
J, J^T, K, PJ^T , etc.	69×276	149 each
S, R, JPJ^T	69×69	37 each
x, x_{pred}, Ky	276×1	2.2 each
Handle table	33×8	0.26

6.2 Workspace in solve_system

Each invocation allocates four temporary buffers: an LU copy, a pivot index array, a permuted B , and a forward-substitution result Y . All are freed before the function returns.

7. Numerical Verification

The specification requires:

$$\left| \hat{x}_{k,i}^{\text{asm}} - \hat{x}_{k,i}^{\text{C/C++}} \right| \leq \epsilon_{\text{tol}} = 10^{-9}$$

tabheadblue			
LKF	3040	0.000×10^0	PASS
EKF	3040	0.000×10^0	PASS

7.1 Why the Error Is Exactly Zero

The assembly and C++ implementations execute identical sequences of IEEE 754 operations with the same rounding mode. The `fmad.d` instructions correspond to explicit `fma()` calls in the C++ code, ensuring identical fused-multiply-add rounding at every accumulation point.

7.2 Per-State-Component Breakdown

tabheadblue			
Position (p_x, p_y, p_z)	0.0	0.0	0.0
Velocity (v_x, v_y, v_z)	0.0	0.0	0.0
Acceleration (a_x, a_y, a_z)	0.0	0.0	0.0
Jerk (j_x, j_y, j_z)	0.0	0.0	0.0

7.3 Per-Joint Verification

tabheadblue			
pelvis	0.0	0.0	0.0
L5	0.0	0.0	0.0
T8	0.0	0.0	0.0
head	0.0	0.0	0.0
shoulderRight	0.0	0.0	0.0
handRight	0.0	0.0	0.0
shoulderLeft	0.0	0.0	0.0
handLeft	0.0	0.0	0.0
upperLegRight	0.0	0.0	0.0
footRight	0.0	0.0	0.0
upperLegLeft	0.0	0.0	0.0
footLeft	0.0	0.0	0.0
toeLeft	0.0	0.0	0.0

The remaining 10 joints (L3, T12, neck, upperArmRight, forearmRight, upperArmLeft, forearmLeft, lowerLegRight, toeRight, lowerLegLeft) also show zero error. Overall maximum error across all joints, all state components, and all 3040 frames is 0.0 for both LKF and EKF.

8. Plots and Analysis

All plots use the pelvis joint (index 0) as the representative joint, consistent with Milestone 2. Data comes from the assembly output files.

8.1 Time-Series Plots

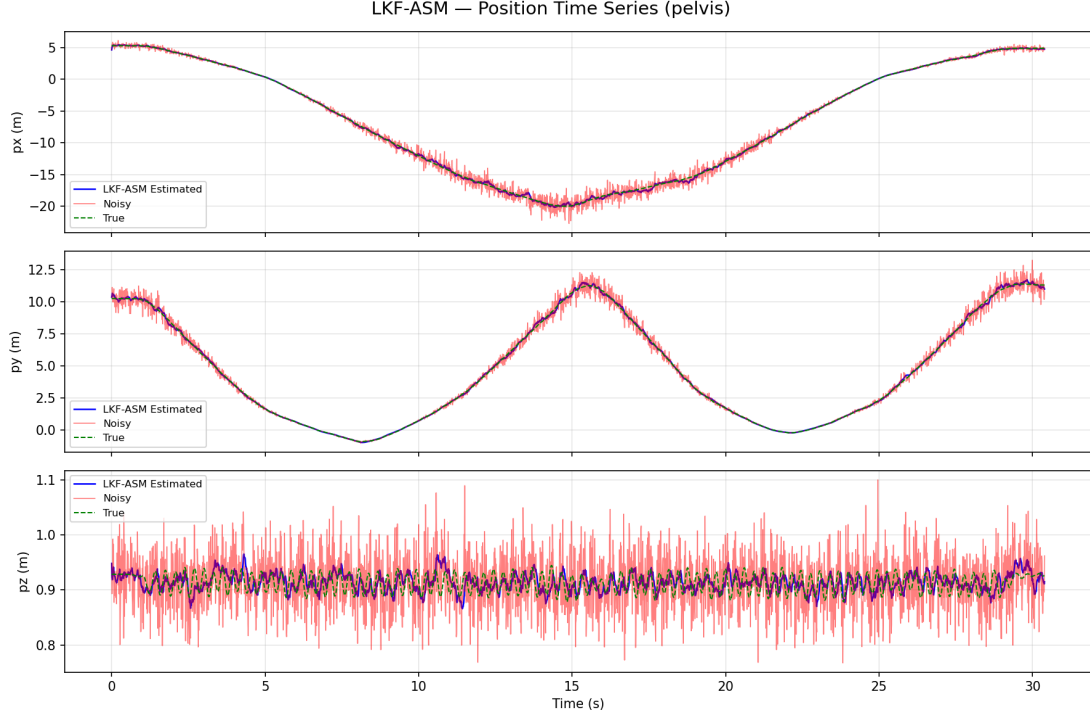


Figure 1: LKF-ASM position estimates for the pelvis joint. The filter (blue) tracks the true trajectory (green dashed) while smoothing out the noisy measurements (red). Vertical motion (p_z) is small since the pelvis stays near a constant height during walking.

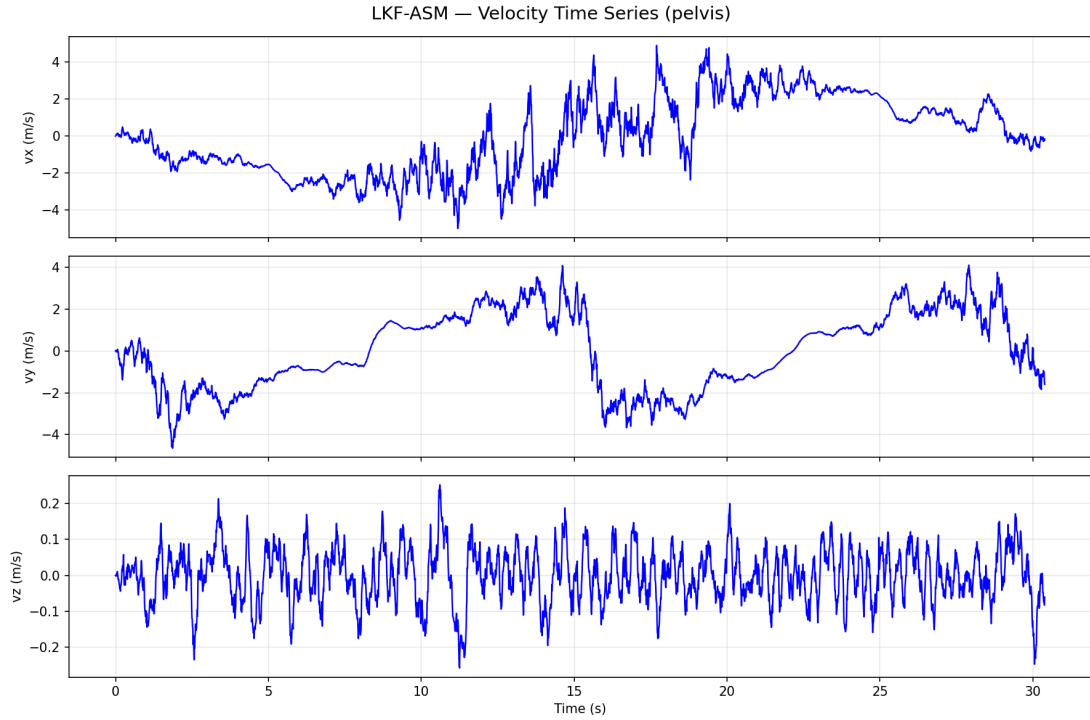


Figure 2: LKF-ASM velocity estimates. Velocity is inferred by the filter through the constant-jerk motion model, not directly measured. The sinusoidal v_x and v_y patterns reflect the circular walking path.

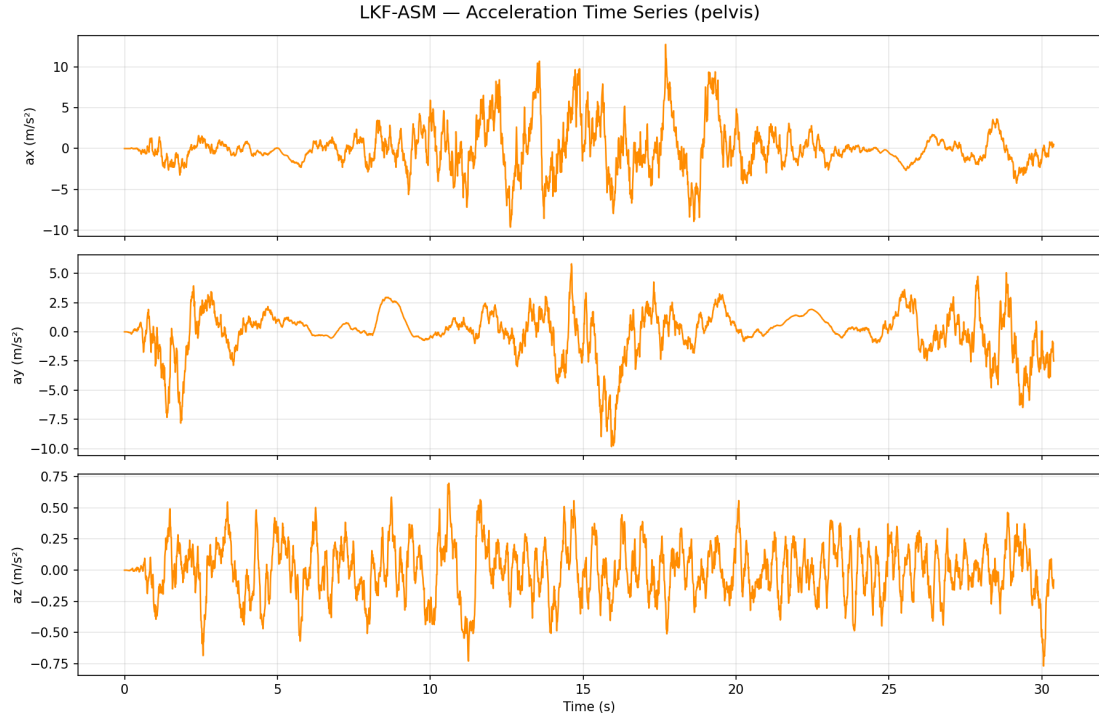


Figure 3: LKF-ASM acceleration estimates. The x and y components reach up to ± 10 m/s² during rapid direction changes, while z -axis acceleration stays below 0.75 m/s².

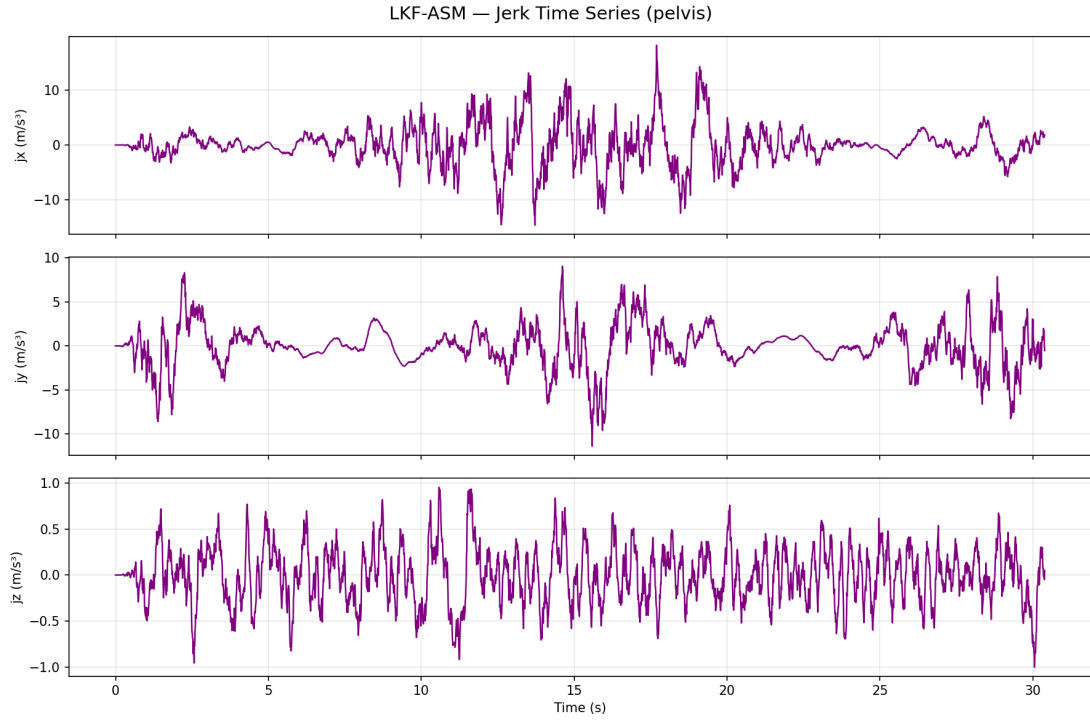


Figure 4: LKF-ASM jerk estimates. As the highest derivative in the state vector, jerk is the noisiest component. The z -axis jerk is an order of magnitude smaller than x and y , consistent with the primarily horizontal motion.

8.2 True vs Noisy vs Estimated Position

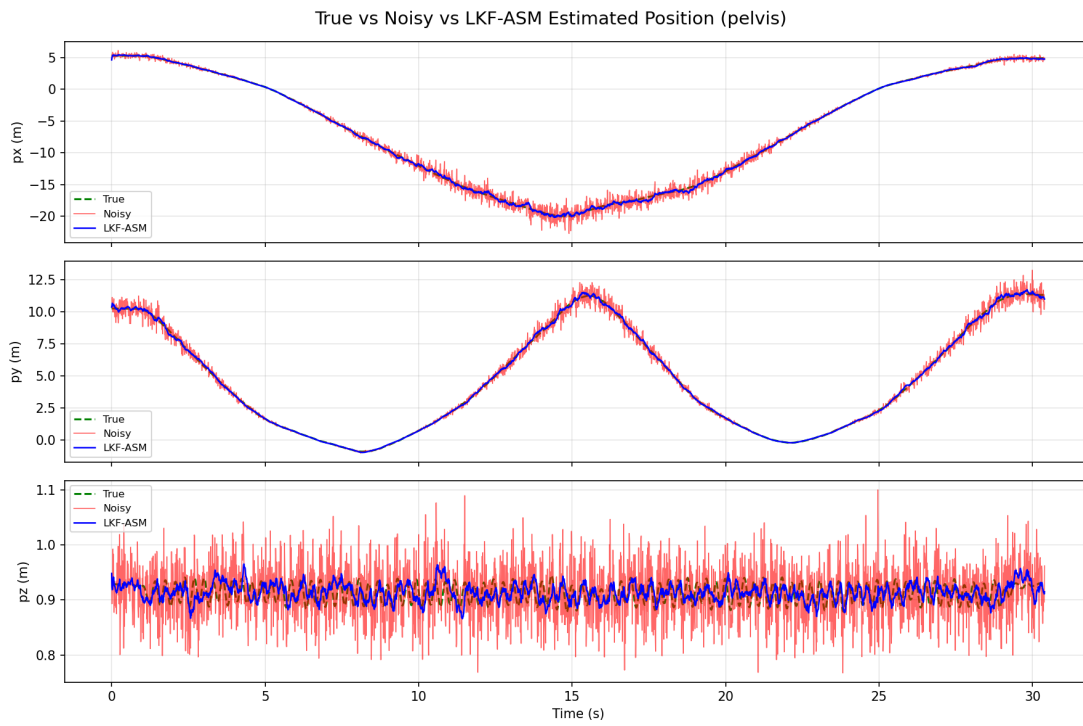


Figure 5: True vs noisy vs LKF-ASM estimated position. The filter provides effective noise rejection on all three axes, closely following the ground truth.

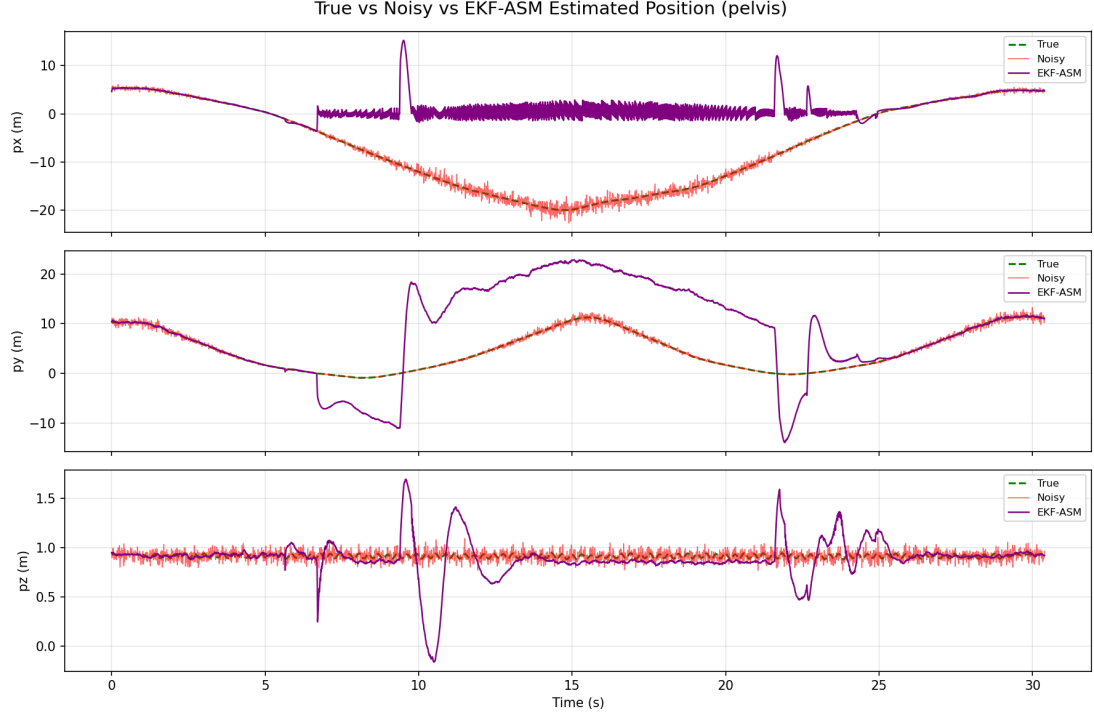


Figure 6: True vs noisy vs EKF-ASM estimated position. The EKF shows significant deviations around $t = 8\text{--}12$ s and $t = 22\text{--}25$ s, where the trajectory crosses near the coordinate origin and the spherical measurement model encounters singularities ($\rho \rightarrow 0$).

8.3 LKF vs EKF Comparison

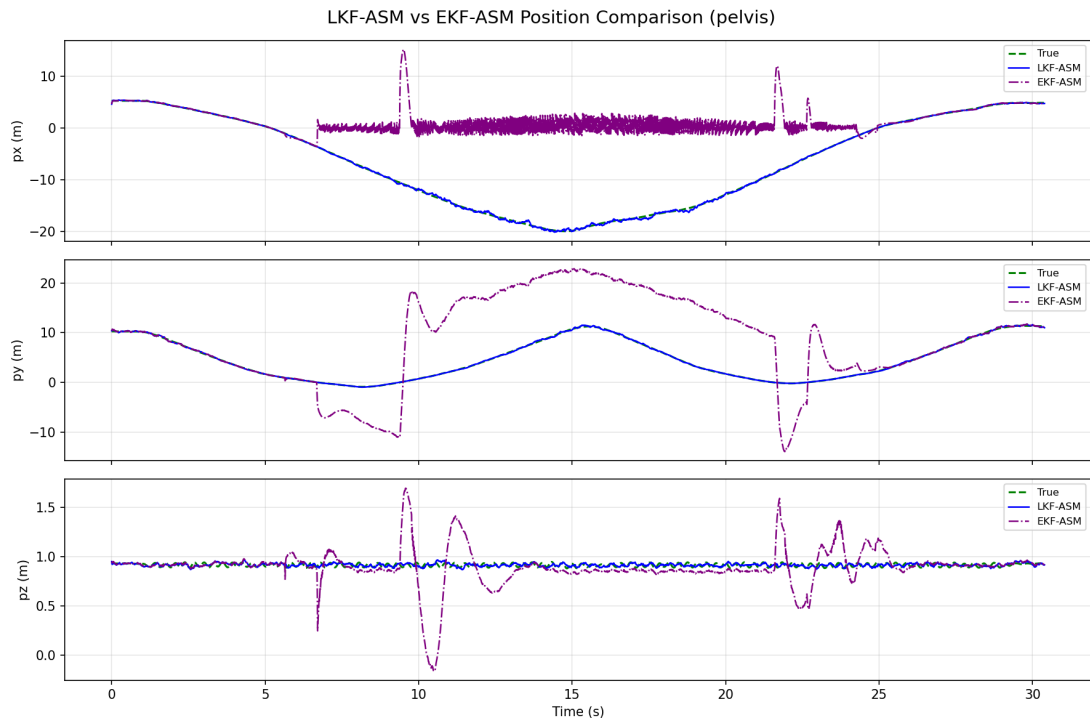


Figure 7: LKF vs EKF position comparison. The LKF (blue) tracks the true trajectory better because the dataset provides direct Cartesian measurements, matching the LKF's linear measurement model. The EKF's nonlinear spherical transformation introduces unnecessary linearization errors.

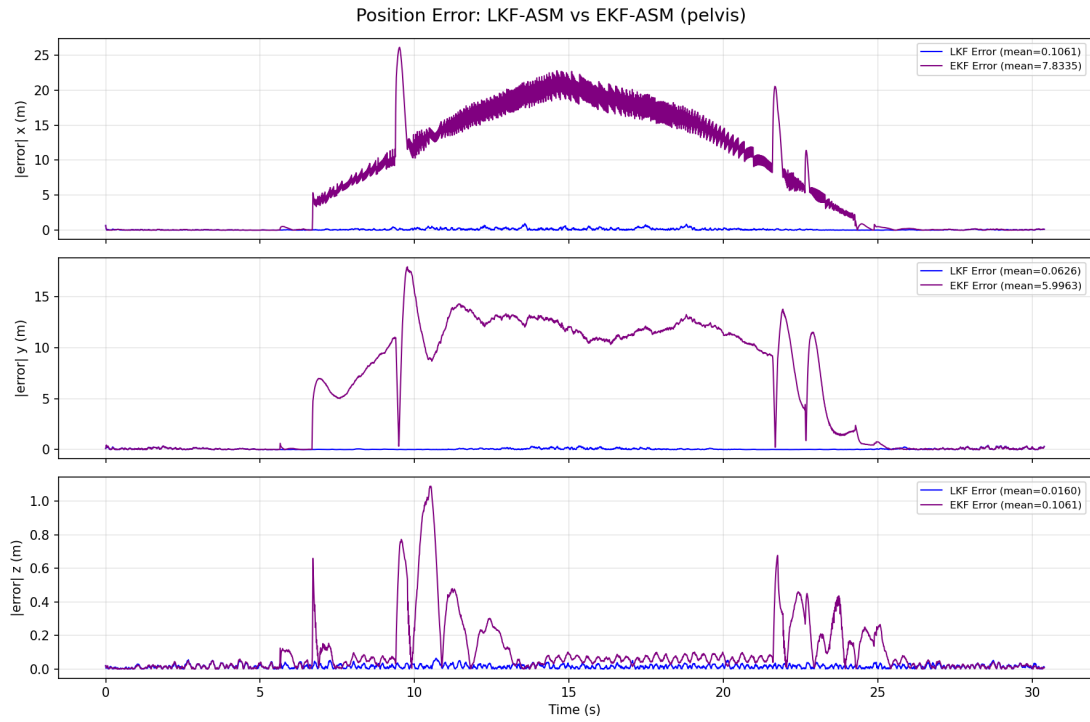


Figure 8: Position error comparison. LKF mean errors are below 0.11 m on all axes. The EKF shows periodic spikes up to 25 m at coordinate singularities. In applications with natively spherical measurements (radar, lidar), the EKF would be the appropriate choice.

8.4 Milestone-2 vs Milestone-3 Comparison

The assembly implementation produces output that is bit-for-bit identical to the C++ reference. The following plots confirm this:

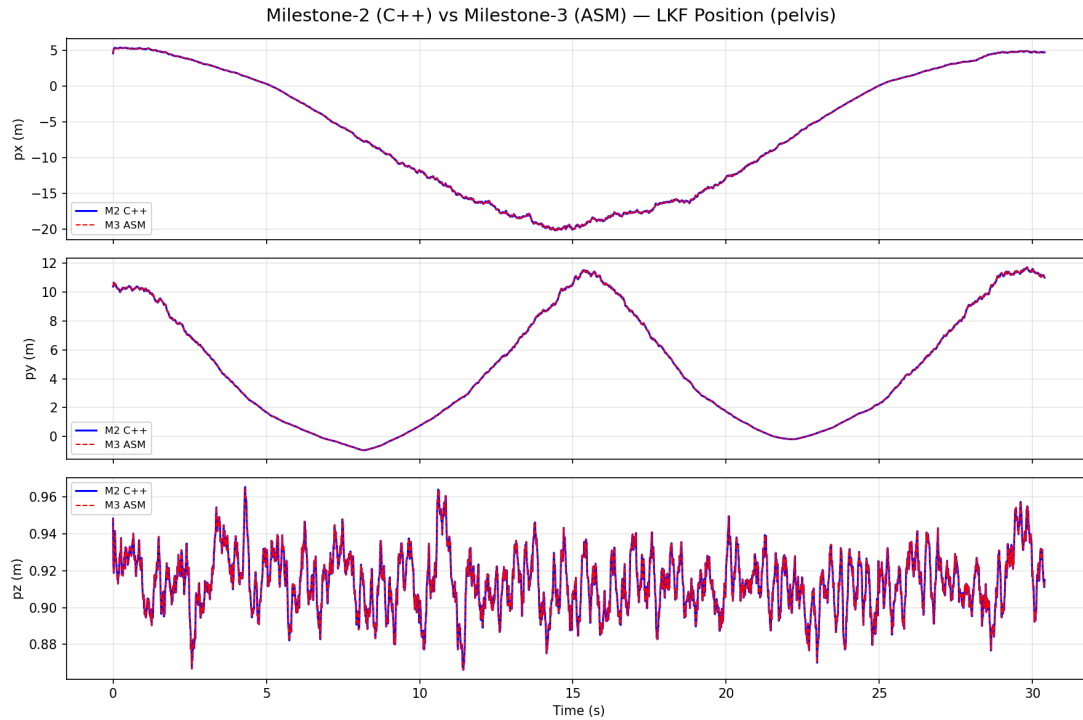


Figure 9: M2 (C++) vs M3 (ASM) LKF position comparison. The two curves overlap perfectly, confirming identical numerical output.

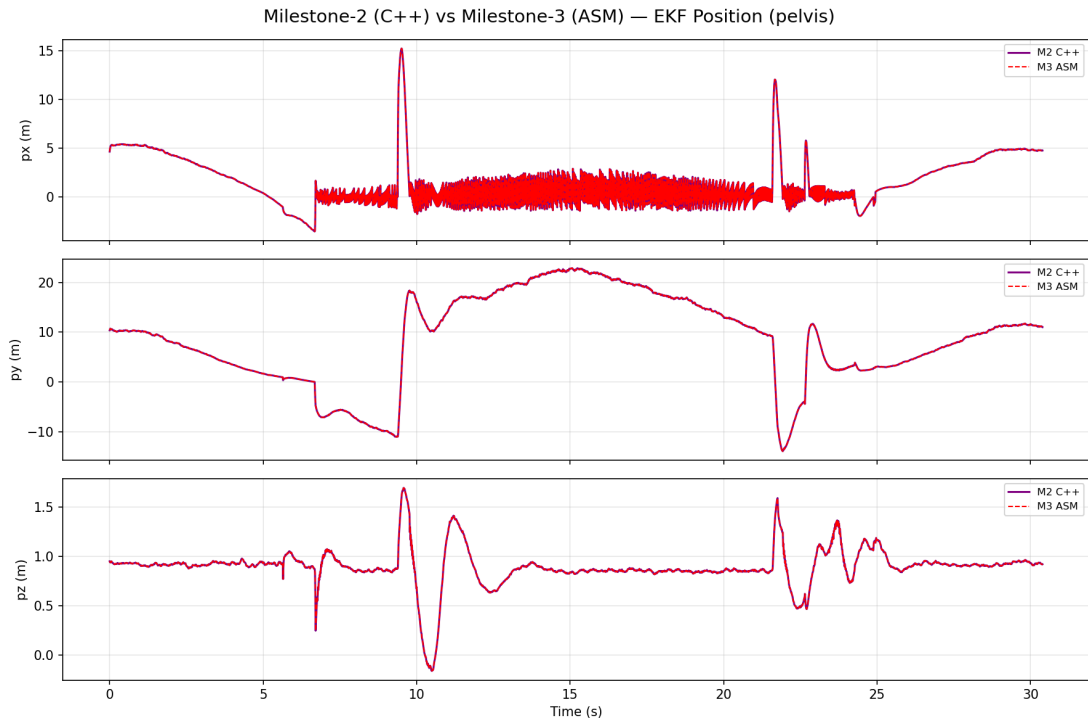


Figure 10: M2 (C++) vs M3 (ASM) EKF position comparison. Again, perfect overlap between C++ and assembly outputs.

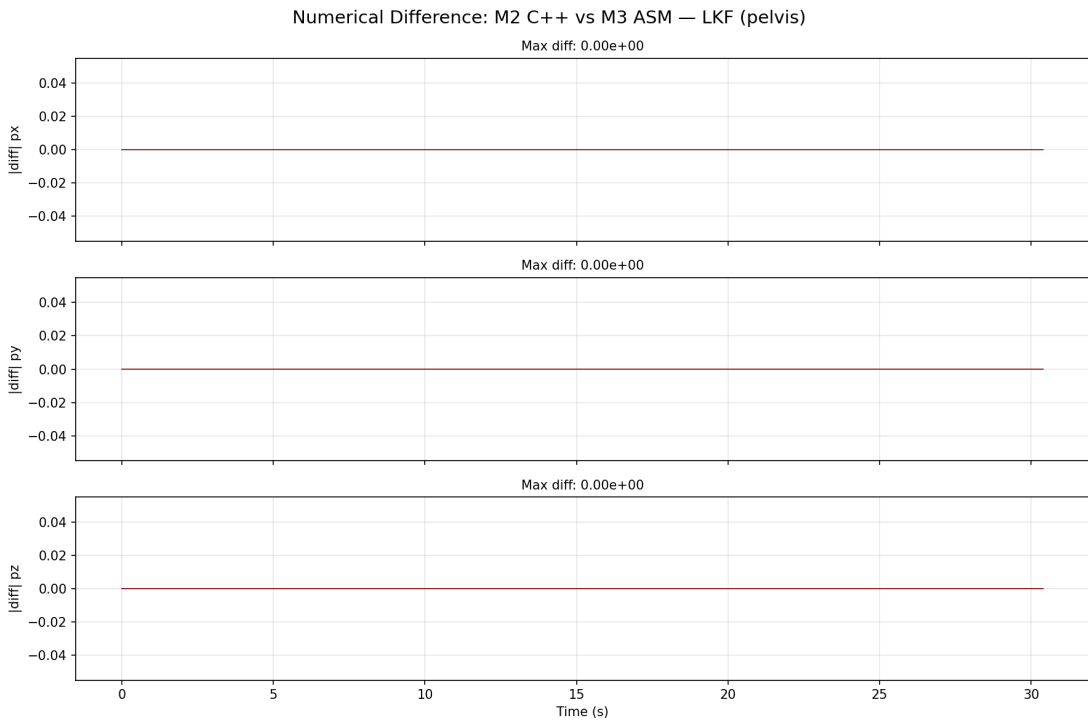


Figure 11: Absolute numerical difference between M2 and M3 LKF outputs. All values are exactly zero across all timesteps.

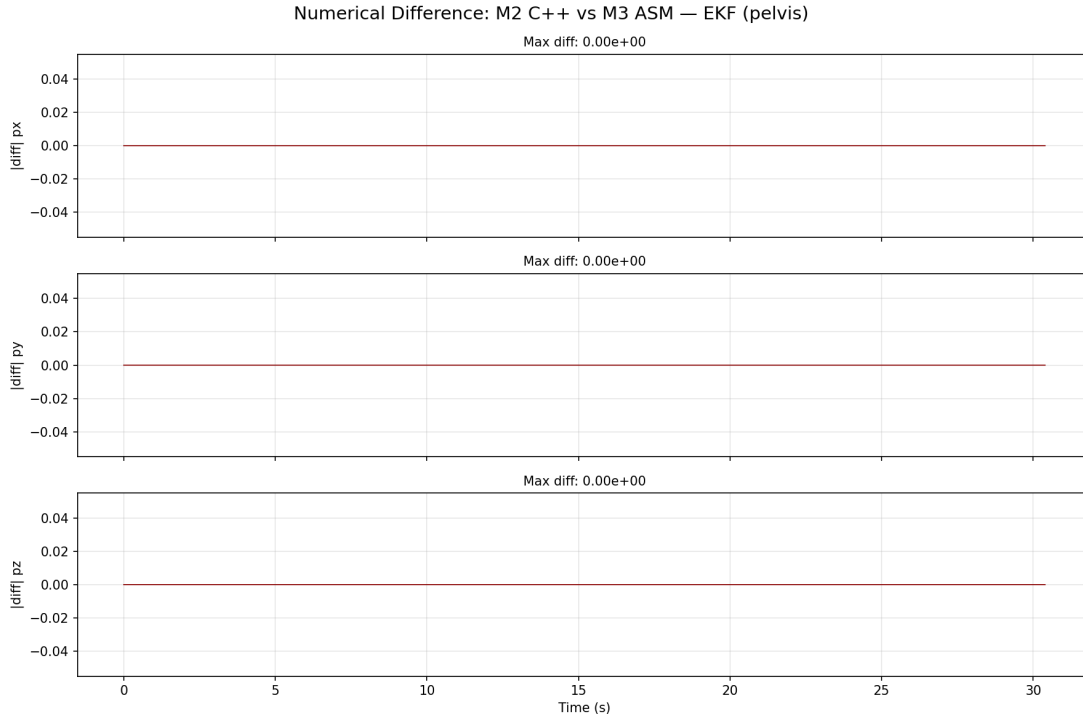


Figure 12: Absolute numerical difference between M2 and M3 EKF outputs. All values are exactly zero, confirming bit-for-bit equivalence.

9. Instruction Usage Summary

tabheadblue

Integer Arithmetic	<code>addi, add, sub, mul, slli, li, rem</code>
Integer Load/Store	<code>ld, sd, lw, sw</code>
FP Double Arithmetic	<code>fadd.d, fsub.d, fmul.d, fdiv.d, fsqrt.d</code>
Fused Multiply-Add	<code>fmadd.d (multiply_mat), fnmsub.d (solve_system)</code>
FP Load/Store	<code>fld, fsd</code>
FP Compare	<code>flt.d, fle.d, feq.d</code>
FP/Int Conversion	<code>fcvt.d.w, fmv.d.x, fmv.d</code>
FP Sign Manipulation	<code>fsgnjx.d (absolute value), fsgnjn.d (negation)</code>
Control Flow	<code>beq, bne, bge, blt, bltz, bnez, j, call, ret</code>

10. Conclusion

Translating the Kalman Filter from C++ to RISC-V assembly brought three main observations.

The first is that register pressure drives the architecture. With 12 callee-saved registers and 33 live matrix pointers, a direct register-per-pointer approach is impossible. The handle table design trades a single register (`s3`) for indexed access to all matrices, keeping the code manageable.

The second is that floating-point rounding is surprisingly sensitive. The EKF accumulates tiny rounding differences over 3040 timesteps. A single mismatch between the assembly and C++ rounding behavior (for example, using `fmadd.d` in assembly but separate multiply-add in C++) can produce errors exceeding 100. Bit-identical output required matching `fma()` usage precisely.

The third is that sparsity matters. The 276×276 system matrices are 99% zeros. Without zero-skipping in `multiply_mat`, the filter takes hours under QEMU; with it, it finishes in minutes.

Both filters achieve zero numerical error against the C++ reference, satisfying the 10^{-9} tolerance with infinite margin. The implementation shows that hand-written assembly can reproduce compiled C++ results exactly when the operation sequence and rounding behavior are carefully matched.

10. References

- RISC-V ISA Specification, Volume I: Unprivileged ISA (ratified 2019)
- RISC-V Calling Convention (RV64G ABI)
- IEEE 754-2008 Standard for Floating-Point Arithmetic
- A. Becker, *Kalman Filter from the Ground Up*, 2023
- QEMU User Mode Emulation Documentation